

Towards Trustworthy Sentiment Analysis in Software Engineering: Dataset Characteristics and Tool Selection

Martin Obaidi[✉], Marc Herrmann[✉], Kurt Schneider[✉]

Leibniz University Hannover, Software Engineering Group
Hannover, Germany

{martin.obaidi, marc.herrmann, kurt.schneider}@inf.uni-hannover.de

Jil Klünder[✉]

University of Applied Sciences
FHDW Hannover
Hannover, Germany
jil.kluender@fhdw.de

Abstract—Software development relies heavily on text-based communication, making sentiment analysis a valuable tool for understanding team dynamics and supporting trustworthy AI-driven analytics in requirements engineering. However, existing sentiment analysis tools often perform inconsistently across datasets from different platforms, due to variations in communication style and content.

In this study, we analyze linguistic and statistical features of 10 developer communication datasets from five platforms and evaluate the performance of 14 sentiment analysis tools. Based on these results, we propose a mapping approach and questionnaire that recommends suitable sentiment analysis tools for new datasets, using their characteristic features as input.

Our results show that dataset characteristics can be leveraged to improve tool selection, as platforms differ substantially in both linguistic and statistical properties. While transformer-based models such as SetFit and RoBERTa consistently achieve strong results, tool effectiveness remains context-dependent. Our approach supports researchers and practitioners in selecting trustworthy tools for sentiment analysis in software engineering, while highlighting the need for ongoing evaluation as communication contexts evolve.

Index Terms—Social Software Engineering, Sentiment Analysis, Software Projects

I. INTRODUCTION

Sentiment analysis is an increasingly important tool in software engineering (SE) [1], [2], [3], as it supports understanding and improving team communication and collaboration [4]. Modern SE relies on textual communication, e.g., in issue trackers or chat tools such as Slack [5], [6], prompting the development of sentiment analysis tools tailored to developer interactions [6], [7], [8], [9]. These tools are typically based on machine learning and are trained using datasets labeled with sentiment polarity (positive, negative, neutral). Such datasets reflect technical language and platform-specific conventions, which differ significantly from everyday user-generated text [10], [11]. Understanding the nuances of developer communication has become integral to software process analysis, especially as teams adopt new collaborative practices and AI-powered tooling.

A major challenge is the variability of tool performance across datasets and contexts [12], [13], [14], [15]. Tools trained on one platform (e.g., GitHub) often perform poorly when

applied to data from others (e.g., Jira), and combining datasets or tools has not consistently yielded better results [13], [16]. Lexicon-based approaches can be more robust but lack domain nuance [12]. These performance differences are closely linked to dataset characteristics, such as specific linguistic features (e.g., politeness, technical focus) and statistical attributes (e.g., document length, emoticon frequency) [13], [17]. For instance, Stack Overflow posts are often concise and factual, while Jira discussions tend to be more detailed and collaborative [17].

For researchers and practitioners seeking to analyze new, unlabeled communication data (e.g., [18], [19], [11]), the question arises: Which sentiment analysis tool is most appropriate for the dataset at hand? Comparing dataset characteristics to those of existing, labeled datasets can help to inform this choice, especially as retraining or building new tools is often infeasible.

Despite the relevance of this challenge, no prior work has systematically mapped the linguistic and statistical characteristics of SE datasets to the performance of sentiment analysis tools, or provided practical guidance for tool selection in the face of dataset variability. In this paper, we address this gap with the following contributions:

- An analysis of ten labeled datasets from five SE platforms, identifying key linguistic and statistical characteristics relevant for sentiment analysis.
- A comprehensive evaluation of 14 sentiment analysis tools, reporting their performance on each dataset.
- A mapping between dataset characteristics and tool performance, enabling informed recommendations for tool selection.
- A questionnaire that supports practical tool recommendations for new, unlabeled datasets based on their communication characteristics.

Our approach enables practitioners and researchers to select sentiment analysis tools that are well-matched to their data, supporting more trustworthy and explainable AI-based analysis of software communication. Rather than requiring detailed linguistic annotation of new datasets, our questionnaire allows users to make informed, subjective assessments based on

empirically derived features.

The remainder of this paper is structured as follows: Section II reviews related work. Section III presents our study design. Section IV reports our results, and Section V discusses implications and limitations. Section VI concludes.

II. BACKGROUND AND RELATED WORK

Sentiment analysis has become central in software engineering (SE) research, enabling the study of communication, collaboration, and well-being in development teams [3], [1], [2]. In this section, we discuss prior work on (a) sentiment datasets and their characteristics, (b) sentiment analysis tools in SE, and (c) recent findings regarding subjectivity and trustworthy interpretation in team settings.

A. Datasets for Sentiment Analysis in SE

A range of labeled datasets for sentiment analysis in SE have been developed, each reflecting the linguistic and contextual nuances of different platforms. For instance, Stack Overflow datasets [5], [20], [15] typically consist of concise, technical posts labeled for sentiment polarity by multiple raters, while datasets from GitHub [21], [22], [23] capture a mix of issue comments and code reviews. Jira datasets [24], [25] focus on collaborative problem solving and are often labeled with both emotions and polarity. Code review datasets [9], [23] and app review datasets [15] reflect further diversity in communication style, granularity, and annotation schemes.

Dataset construction practices vary significantly: some use emotion annotation later mapped to sentiment polarity, while others rely directly on polarity labels. Annotation is often performed by multiple raters, but inter-rater agreement remains a challenge [26]. Moreover, platform-specific jargon and evolving communication styles complicate generalization across datasets.

B. Sentiment Analysis Tools in SE

Early sentiment analysis in SE relied on lexicon-based or classical machine learning tools, such as SentiStrength [27], SentiStrength-SE [8], and Senti4SD [6]. These tools incorporate domain-specific dictionaries or heuristics, enabling polarity classification of developer communication without the need for retraining. More recent approaches employ supervised machine learning and deep learning: SentiCR [9] and SentiSW [22] use features such as TF-IDF and gradient boosting, while transformer-based models like BERT and RoBERTa have demonstrated strong results when fine-tuned on SE data [28], [16]. Novel techniques also leverage emoji-based supervision [29] or ensemble/voting schemes [7], [30].

Recent benchmarks highlight the continued evolution in this field, with large language models (LLMs) such as GPT and zero-shot/few-shot learning approaches increasingly evaluated for sentiment analysis in SE [31]. However, tool performance remains highly sensitive to dataset characteristics and annotation schemes, motivating the need for more principled recommendations.

C. Trustworthiness, Subjectivity, and Sentiment Perception

A recurring challenge is the subjectivity of sentiment annotation and perception among developers. Recent studies reveal substantial variability in human sentiment judgments: Herrmann et al. [32] found distinct developer groups with differing perceptions, independent of demographics, and showed via simulation [33] that incomplete team mood surveys can distort aggregate sentiment. Annotation agreement with gold labels in SE sentiment datasets can be as low as 62.5% [26], [34], underlining the difficulty of obtaining reliable ground truth.

These findings emphasize the need for trustworthy, explainable, and context-aware sentiment analysis in SE—an essential goal when integrating such tools in critical, human-centric AI pipelines as highlighted in RETRAI. Tool evaluation and recommendations must account for both the technical characteristics of datasets and the inherent variability in human perception.

D. Gaps and Motivation

While many tools and datasets exist, few studies systematically analyze how linguistic and statistical dataset characteristics affect tool performance, or offer practical guidance for tool selection across diverse contexts. In particular, the mapping between dataset properties and trustworthy tool recommendations remains an open research challenge, with significant implications for explainable AI and requirements engineering in SE.

III. STUDY DESIGN

This section outlines our selection of tools and datasets, the approach for analyzing dataset characteristics and tool performance, and our mapping procedure for tool recommendation. Figure 1 provides an overview.

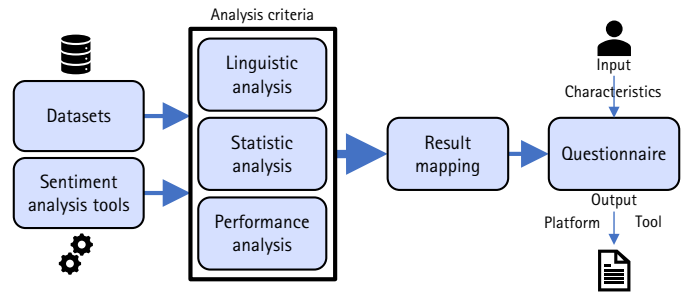


Fig. 1: Overview of the study design.

To recommend suitable sentiment analysis tools for new software engineering (SE) datasets, we first analyze linguistic and statistical characteristics of diverse datasets, then assess the performance of sentiment analysis tools, and finally map these insights to a practical questionnaire for tool recommendation.

A. Research Questions

Our work is guided by the following research questions:
RQ1: *How does the communication of SE platforms differ?*

RQ2: How do sentiment analysis tools perform on communication data from different SE platforms?

RQ3: What is the best sentiment analysis tool for a specific software project?

B. Data

We selected sentiment analysis datasets from a recent systematic literature review [3], focusing on datasets with polarity labels (positive, negative, neutral). If emotion labels could be reliably mapped to polarity using established heuristics [12], we included them as well. Table I summarizes the datasets and their class distributions.

TABLE I: Overview of the used datasets. The symbol “#” means “number of” and “Docs” stands for “documents”. *The class neutral in Code Reviews is “non-negative”. n stands for the sample size with equal or greater than 95% confidence interval and an error rate of 5%.

Platform	Dataset	#Docs	#Neg	#Neu	#Pos	n
App Reviews	App [15]	341	130	25	186	193
Code Reviews	Code [9]	1600	398	1202*	0*	310
GitHub	GH 1 [21]	7122	2087	3022	2013	-
	GH 2 [22]	2962	401	1968	593	-
	GH 3 [23]	585	73	419	93	-
	$\sum$	10669	2561	5409	2699	371
Jira	Jira 1 [8]	1795	541	616	638	-
	Jira 2 [24]	6300	778	4165	1357	-
	$\sum$	8095	1319	4781	1995	367
Stack Overflow	SO 1 [15]	1500	178	1191	131	-
	SO 2 [5]	4687	1145	1947	1595	-
	SO 3 [20]	4518	496	3132	890	-
	$\sum$	10705	1819	6270	2616	371

When necessary, we mapped emotion annotations to polarity using heuristics from prior work [12], while acknowledging potential limitations. For instance, in Jira datasets, emotions such as “Excited” were mapped to positive, while “Stress” was mapped to negative. In ambiguous cases, only documents with clear mapping were included. The neutral class was operationalized as documents with neutral or no strong valence.

For each platform, all datasets were pooled, and a representative random sample was drawn (minimum sample size for 95% confidence and 5% error). Sampling preserved original polarity distributions except for App Reviews, where all neutral items were retained for adequate class representation.

1) *Data Pre-processing:* Datasets were pre-processed to remove entries irrelevant for polarity classification (e.g., sarcasm in GH 3 [23]), to harmonize column structures, and to map emotion labels where needed. Details of initial processing can be found in the respective dataset papers.

C. Sentiment Analysis Tools

We selected 14 representative sentiment analysis tools, covering transformer models (e.g., BERT [35], RoBERTa, ELECTRA, SetFit [36], XLNet) and established SE-specific tools (e.g., Senti4SD [6], SentiCR [9], SentiStrength [27], SentiStrength-SE [8], SentiSW [22], Senti-Analyzer [30]).

Both lexicon-based and machine learning methods are included for breadth.

Tools were used as described in their respective publications, with adaptation to our class labels if needed. For tools with pre-defined train/test splits (e.g., SentiCR, Senti4SD), we avoided data leakage by retraining solely on our training splits (80/20), never reusing the original splits.

D. Performance Metrics

We evaluated tool performance using micro- and macro-averaged F_1 scores, as well as their average (overall score), following established practice [12], [14]. The micro F_1 reflects accuracy, while macro F_1 balances across classes. This dual perspective mitigates the impact of class imbalance. Performance is reported under consistent, controlled conditions, and findings are interpreted as empirical guidance rather than definitive rankings, given the inherent context-sensitivity of sentiment classification in SE.

E. Interrater Agreement

To assess the reliability of our manual linguistic feature coding, we computed Fleiss’ κ [37] to quantify agreement among the three raters (researchers). Agreement was interpreted using the categories from Landis and Koch [38]. For additional transparency, we report both κ and raw agreement percentages (e.g., 60% agreement if raters agree on 6 out of 10 items).

F. Platform Analysis Approach

1) *Inputs:* Our analysis is based on the datasets (Section III-B) and sentiment analysis tools (Section III-C). Each dataset was analyzed for linguistic features and statistical values, and each tool was evaluated on all datasets for performance.

2) *Linguistic Features:* We employed a multi-step, collaborative coding process to identify linguistic features characteristic of SE communications across platforms (see Figure 2). Initially, three researchers jointly coded a small, balanced sample (15 positive, 15 neutral, 15 negative statements per platform), iteratively refining a shared feature list. Then, each researcher independently coded additional samples (10 per polarity per platform), updating the feature set as needed.

All identified linguistic features were subsequently grouped into categories. The full platform samples were then labeled by all three researchers, enabling calculation of Fleiss’ κ for interrater reliability and resolving disagreements via majority vote. For each platform, the frequency of each linguistic feature was computed as the percentage of documents exhibiting that feature.

3) *Statistics:* In addition to linguistic features, we computed eight statistical values for each sample: (1) average number of characters per document, (2) characters per word, (3) words per document, (4) capitalized words, (5) spelling mistakes (via `pyspellchecker`), (6) emoticons, (7) question marks, and (8) exclamation marks. These metrics are standard in prior work [39], [40] and provide further insight into dataset characteristics.

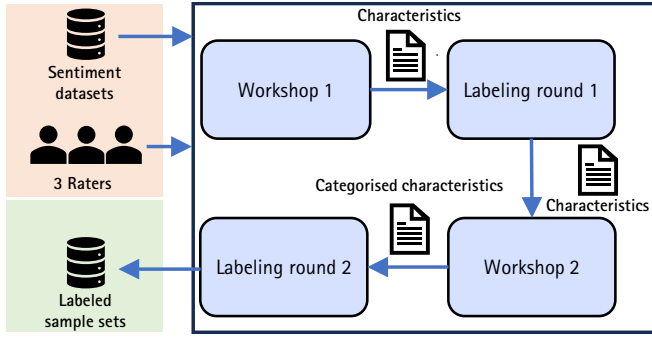


Fig. 2: Steps of linguistic feature coding and platform analysis.

TABLE II: Answer options and corresponding frequency intervals

Answer Option	Interval
Fully true (True)	75–100%
More likely to be true (Likely)	50–75%
More unlikely to be true (Unlikely)	25–50%
Not true at all (Untrue)	0–25%
Not specified	—

4) *Performance*: For tool evaluation, each dataset was split into 80% training and 20% test sets (for ML-based tools). Dictionary-based tools were evaluated on the full test set. We computed micro- and macro-averaged F_1 scores, and their mean (overall score), using the `classification_report` from Scikit-learn¹. This process allowed us to empirically identify the best-performing tool for each dataset, supporting our final tool recommendation mapping.

5) *Result Mapping & Questionnaire*: After analyzing dataset characteristics and tool performance, we map these results to user input via a structured questionnaire. This mapping is essential, as communication styles evolve and may differ even within the same platform. Our approach enables users to compare their own (potentially new or proprietary) datasets to existing SE communication data, supporting robust tool recommendations even in cases of dataset drift or domain adaptation.

a) *Mapping Linguistic Features*.: Linguistic feature frequencies (as percentages) are divided into four intervals, each corresponding to a Likert-style answer (see Table II). Users indicate, for each linguistic feature, whether it is “Fully true,” “More likely to be true,” “More unlikely to be true,” or “Not true at all” for their dataset. This supports subjective, yet structured, mapping without requiring users to calculate exact frequencies.

Each platform is mapped to an interval per feature. If a user’s answer matches the interval for a platform, that platform receives a point. If no platform fits the interval or the user selects “Not specified,” the feature is mapped as “Ambiguous.”

b) *Mapping Statistical Values*.: Since statistical values (e.g., average words per document) are absolute numbers, we

calculate the absolute difference between the user’s input and each platform’s value, assigning a point to the platform with the smallest difference for each statistic. This ensures a flexible matching even for out-of-distribution cases.

c) *Recommendation Output*.: After mapping all features and statistics, the platform with the highest score is recommended, along with its empirically best-performing sentiment analysis tool. In the case of a tie, all tied platforms and corresponding tools are recommended. If most answers are “Not specified” or if no platform matches the majority of features, the dataset is considered “Ambiguous.” In this case, we recommend both SetFit [36] and the dictionary-based SentiStrength-SE [8], which demonstrated robust, cross-platform performance.

This approach allows practical, low-barrier tool selection without requiring in-depth NLP expertise, and supports robust handling of uncertainty or novel datasets—aligning with trustworthy, adaptable AI practices.

IV. RESULTS

This section presents the results of our platform and tool performance analyses.

A. Linguistic Feature Analysis

We identified 13 linguistic features relevant to SE communication, summarized in Table III. Each document in the sample sets was coded for the presence or absence of these features.

TABLE III: Linguistic features identified in SE communication

Feature	Description
1. Direct emotion	Explicit expression of emotions
2. Emphasized positivity	Strongly expressed positive opinions
3. Technical focus	Predominantly technical content
4. Balanced critique	Negative comments with positive points included
5. Progress sharing	Reporting on project progress or achievements
6. Gratitude	Expressing thanks (distinct from compliments)
7. Inquisitive	Asking about code-level decisions
8. Help seeking/offering	Requests for, or offers of, help
9. Compliments	Compliments on others’ work (distinct from gratitude)
10. Bug fix requests	Explicit requests for bug fixes
11. Constructive criticism	Negative feedback with justification or advice
12. Username mentioning	Mentioning users by handle
13. Name mentioning	Mentioning users by name

Features 1, 2, 6, and 9 relate to emotion and opinion expression; 4 and 11 to feedback dynamics; 3, 5, and 7 to content nature; 8 and 10 to assistance and problem-solving; and 12/13 to personal addressing.

a) *Interrater Agreement*.: Table IV summarizes interrater reliability for coding the 13 features, as measured by Fleiss’ κ and raw agreement. Across all platforms and features, agreement ranged from substantial to almost perfect, supporting the reliability of the coding process.

Table V shows considerable variation in linguistic features across platforms. For instance, features like *Direct emotion*, *Technical focus*, and *Compliments* show wide ranges (over 50 percentage points), while other features (e.g., *Inquisitive*, *Username Mentioning*) exhibit less variation. Thus, some

¹https://scikit-learn.org/stable/modules/generated/sklearn.metrics.classification_report.html

TABLE IV: Interrater agreement: Fleiss’ κ and agreement rates for feature coding

Ling	App		Code		GH		Jira		SO	
	κ	Agr	κ	Agr	κ	Agr	κ	Agr	κ	Agr
L_1	0.84	0.89	0.90	0.98	0.89	0.96	0.88	0.96	0.88	0.97
L_2	0.81	0.88	1.00	1.00	0.79	0.91	0.84	0.98	0.89	0.98
L_3	—	1.00	0.82	0.86	0.76	0.85	0.73	0.81	0.90	0.94
L_4	0.88	0.93	—	1.00	—	1.00	—	1.00	0.87	0.98
L_5	0.91	0.99	—	1.00	0.86	0.97	0.82	0.93	0.93	1.00
L_6	1.00	1.00	0.94	1.00	0.97	0.94	0.97	0.98	1.00	1.00
L_7	—	1.00	0.77	0.96	0.77	0.98	1.00	1.00	1.00	1.00
L_8	0.94	0.99	0.84	0.90	0.82	0.85	0.89	0.95	0.89	0.92
L_9	0.81	0.87	—	1.00	0.97	0.97	0.90	0.96	0.82	0.92
L_{10}	0.90	0.96	1.00	1.00	1.00	1.00	1.00	1.00	—	1.00
L_{11}	0.83	0.88	0.85	0.90	0.80	0.92	0.71	0.87	0.85	0.95
L_{12}	0.75	0.99	1.00	1.00	0.95	0.99	1.00	1.00	0.94	0.99
L_{13}	—	1.00	0.97	1.00	0.86	0.99	0.98	0.99	0.89	0.99

TABLE V: Linguistic feature analysis for each platform. Range: max-min value for each feature. Abbreviations: App = App Reviews, Code = Code Reviews, GH = GitHub, Jira = Jira, SO = Stack Overflow.

Linguistic Feature	App	Code	GH	Jira	SO	Range
Direct emotion	60.6%	7.4%	18.9%	12.8%	11.3%	53.2%
Emphasized positivity	31.3%	0.9%	4.3%	5.6%	7.7%	30.5%
Technical focus	0.0%	49.4%	45.6%	67.6%	70.1%	70.1%
Balanced critique	31.9%	0.0%	0.0%	0.0%	4.8%	31.9%
Progress sharing	2.1%	0.0%	8.4%	15.0%	1.3%	15.0%
Gratitude	8.1%	2.6%	35.1%	57.8%	15.4%	55.2%
Inquisitive	0.0%	6.8%	3.8%	0.3%	0.3%	6.8%
Help Seeking and Offering	3.1%	28.7%	41.5%	17.7%	49.9%	46.8%
Compliments	63.6%	0.0%	16.0%	20.0%	18.7%	63.6%
Bug Fix Requests	17.6%	0.3%	0.5%	0.5%	0.0%	17.6%
Constructive Criticism	66.7%	37.7%	14.6%	20.0%	12.7%	54.0%
Username Mentioning	0.5%	0.6%	11.9%	1.4%	4.6%	11.3%
Name Mentioning	0.0%	3.9%	1.9%	20.7%	3.2%	20.7%

features are highly platform-specific, while others are broadly shared.

Observation: While platforms differ in several linguistic features, similarities remain for certain aspects.

B. Statistical Analysis

In addition to linguistic features, we examined eight statistical attributes per platform (see Table VI).

TABLE VI: Statistical analysis for each platform (averages per document).

Statistic	App	Code	GH	Jira	SO
Characters	176.36	165.74	165.55	104.21	144.39
Characters per word	4.14	4.71	4.53	4.61	4.57
Words	33.15	28.86	27.78	17.43	25.46
Capitalized words	0.00	0.37	0.44	0.22	0.50
Spelling mistakes	0.76	1.98	2.97	1.06	1.38
Emoticons per document	0.02	0.07	0.40	0.14	0.18
Question marks	0.29	0.32	0.24	0.14	0.22
Exclamation marks	0.32	0.03	0.48	0.21	0.27

Distinct differences appear for statistics such as average number of characters (Jira shows shorter entries) and spelling

mistakes (highest in GH). Other measures, such as characters per word, are nearly identical across all platforms.

Observation: Platforms exhibit clear deviations in some statistics, but also share similarities, echoing findings from the linguistic analysis.

C. Performance Analysis

Table VII reports the performance of the 14 sentiment analysis tools across 10 datasets, using micro/macro F_1 and their average (see Section III-D). The highest value per dataset and metric is in bold.

Across most datasets and metrics, SetFit achieves the highest overall score, with RoBERTa and other BERT-based transformer models also performing strongly. For Stack Overflow datasets, SetFit and RoBERTa achieve nearly identical results. Notably, the software engineering-specific tools (e.g., Senti4SD, SentiStrength-SE) do not outperform transformer models in any scenario.

Observation: Transformer-based models such as SetFit and RoBERTa show robust performance across all datasets. SE-specific tools do not outperform these models on any dataset.

D. Questionnaire Mapping

Based on the results from Table V in Section IV-A, we mapped each platform to one of the four intervals described in Section III-F5. Table VIII presents this mapping for each linguistic feature.

Here, “All” means every platform falls into the answer option for that feature; “-” indicates no platform matches, resulting in an ambiguous assignment for that feature. As described in Section III-F5, a point is assigned to the ambiguous dataset if the user selects “Not specified”. In the upper two intervals (“True” and “Likely”), ambiguous assignment is most common.

Table IX gives an example: User input is mapped to intervals, and for each platform it is checked whether it falls into the corresponding interval for that feature. Points are assigned accordingly.

The platform with the most points is suggested as the most similar to the user’s dataset; in this example, *GitHub*. If no clear match exists, or “Not specified” dominates, the ambiguous dataset is returned, and robust cross-platform tools such as SetFit or SentiStrength-SE are suggested.

Observation: The mapping enables identification of the most similar platform and the corresponding recommended tool for a user’s dataset.

V. DISCUSSION

A. Answering the Research Questions

Based on the results from Section IV, we summarize our research questions as follows:

RQ1: There are pronounced differences in linguistic features and statistics across platforms, though some features (e.g.,

TABLE VII: Performance of each tool for each dataset. The best value for each metric and dataset is highlighted in bold.

Dataset	Metrics	ALBERT	BERT	DistilBERT	ELECTRA	RoBERTa	SetFit	XLNet	Senti4SD	SEnti-Analyzer	SentiCR	SentiStrength	SentiStrength-SE	SentiSW
App	Micro averaged F1	0.91	0.93	0.93	0.75	0.91	0.94	0.93	0.87	0.59	0.74	0.62	0.59	0.80
	Macro averaged F1	0.61	0.62	0.63	0.50	0.63	0.64	0.63	0.57	0.53	0.52	0.49	0.50	0.56
	Overall score	0.76	0.78	0.78	0.63	0.77	0.79	0.78	0.72	0.56	0.63	0.55	0.54	0.68
Code	Micro averaged F1	0.87	0.87	0.89	0.86	0.86	0.89	0.82	0.81	0.77	0.79	0.70	0.76	0.78
	Macro averaged F1	0.55	0.80	0.83	0.80	0.82	0.86	0.78	0.74	0.55	0.72	0.58	0.58	0.69
	Overall score	0.71	0.83	0.86	0.83	0.84	0.87	0.80	0.77	0.66	0.75	0.64	0.67	0.74
GH 1	Micro averaged F1	0.92	0.92	0.92	0.92	0.93	0.94	0.92	0.89	0.72	0.82	0.61	0.70	0.78
	Macro averaged F1	0.92	0.92	0.92	0.92	0.93	0.94	0.92	0.90	0.69	0.81	0.62	0.68	0.79
	Overall score	0.92	0.92	0.92	0.92	0.93	0.94	0.92	0.89	0.71	0.81	0.62	0.69	0.79
GH 2	Micro averaged F1	0.84	0.81	0.84	0.80	0.82	0.83	0.76	0.75	0.72	0.74	0.52	0.67	0.74
	Macro averaged F1	0.73	0.72	0.77	0.67	0.72	0.75	0.70	0.61	0.57	0.65	0.52	0.60	0.62
	Overall score	0.78	0.76	0.81	0.74	0.77	0.79	0.73	0.68	0.64	0.69	0.52	0.65	0.68
GH 3	Micro averaged F1	0.69	0.71	0.75	0.71	0.72	0.74	0.67	0.71	0.72	0.68	0.52	0.69	0.67
	Macro averaged F1	0.44	0.28	0.48	0.28	0.52	0.58	0.45	0.41	0.54	0.55	0.50	0.54	0.47
	Overall score	0.57	0.49	0.61	0.49	0.62	0.66	0.56	0.56	0.63	0.61	0.51	0.61	0.57
Jira 1	Micro averaged F1	0.95	0.94	0.96	0.96	0.96	0.97	0.94	0.88	0.79	0.87	0.74	0.83	0.86
	Macro averaged F1	0.94	0.94	0.96	0.96	0.96	0.96	0.93	0.88	0.78	0.85	0.74	0.83	0.86
	Overall score	0.94	0.94	0.96	0.96	0.96	0.97	0.93	0.88	0.78	0.86	0.74	0.83	0.86
Jira 2	Micro averaged F1	0.87	0.85	0.84	0.86	0.86	0.85	0.86	0.81	0.80	0.85	0.65	0.80	0.84
	Macro averaged F1	0.82	0.80	0.79	0.81	0.81	0.80	0.82	0.77	0.74	0.78	0.69	0.79	0.80
	Overall score	0.84	0.82	0.81	0.84	0.84	0.82	0.84	0.79	0.77	0.81	0.67	0.79	0.82
SO 1	Micro averaged F1	0.89	0.86	0.89	0.85	0.89	0.91	0.90	0.86	0.80	0.83	0.69	0.78	0.83
	Macro averaged F1	0.79	0.71	0.77	0.48	0.79	0.80	0.79	0.70	0.46	0.66	0.54	0.42	0.64
	Overall score	0.84	0.79	0.83	0.67	0.84	0.85	0.84	0.78	0.62	0.75	0.61	0.60	0.73
SO 2	Micro averaged F1	0.84	0.82	0.84	0.85	0.87	0.84	0.85	0.82	0.83	0.80	0.76	0.77	0.79
	Macro averaged F1	0.83	0.81	0.83	0.84	0.86	0.83	0.85	0.81	0.83	0.78	0.79	0.78	0.78
	Overall score	0.83	0.81	0.84	0.84	0.87	0.84	0.85	0.81	0.83	0.79	0.78	0.78	0.68
SO 3	Micro averaged F1	0.90	0.88	0.89	0.86	0.90	0.91	0.85	0.83	0.74	0.80	0.65	0.73	0.80
	Macro averaged F1	0.83	0.81	0.85	0.77	0.84	0.86	0.77	0.76	0.56	0.64	0.56	0.52	0.56
	Overall score	0.87	0.85	0.87	0.81	0.87	0.89	0.81	0.80	0.60	0.72	0.60	0.63	0.68

TABLE VIII: Mapping of the platforms to the four answer options for each linguistic feature

L_i	True	Likely	Unlikely	Untrue
L_1	-	App	-	Code, GH, Jira, SO
L_2	-	-	App	Code, GH, Jira, SO
L_3	-	Jira, SO	Code, GH	App
L_4	-	-	App	Code, GH, Jira, SO
L_5	-	-	-	All
L_6	-	Jira	GH	App, Code, SO
L_7	-	-	-	All
L_8	-	-	Code, GH, SO	App, Jira
L_9	-	App	-	Code, GH, Jira, SO
L_{10}	-	-	-	All
L_{11}	-	App	Code	GH, Jira, SO
L_{12}	-	-	-	All
L_{13}	-	-	-	All

TABLE IX: Example of user input for communication linguistic features. The platform *GitHub* receives the highest score (9) and is thus suggested as the output.

L_i	User Input	Interval	App	Code	GH	Jira	SO	Ambig
L_1	Untrue	0-25%	0	1	1	1	1	0
L_2	Untrue	0-25%	0	1	1	1	1	0
L_3	Unlikely	25-50%	0	1	1	0	0	0
L_4	Untrue	0-25%	0	1	1	1	1	0
L_5	Unlikely	25-50%	0	0	0	0	0	1
L_6	Unlikely	25-50%	0	0	1	0	0	0
L_7	Untrue	0-25%	1	1	1	1	1	0
L_8	Unlikely	25-50%	0	1	1	0	1	0
L_9	Untrue	0-25%	0	1	1	1	1	0
L_{10}	N/A	-	0	0	0	0	0	1
L_{11}	Untrue	0-25%	0	0	1	1	1	0
L_{12}	N/A	-	0	0	0	0	0	1
L_{13}	Unlikely	25-50%	0	0	0	0	0	1
Sum	-	-	1	7	9	6	7	4

average word length) are similar. Every linguistic feature shows at least a two-digit range between platforms, indicating

that platform-specific communication styles are distinct and measurable.

RQ2: SetFit [36] achieved the best overall performance for most datasets, with RoBERTa and ELECTRA also leading for certain platforms. However, “best-performing” is highly context-dependent; tool effectiveness varies with dataset characteristics such as communication style, class balance, and domain-specific language. Our results reflect relative performance on the selected datasets and may not generalize to other SE contexts or future datasets.

RQ3: By mapping dataset characteristics to user input via the questionnaire, users can identify which platform is most similar to their data, enabling selection of a tool that has shown strong performance for comparable datasets.

B. Interpretation and Implications

The proposed approach distinguishes platforms based on both linguistic features and simple statistics. Features such as “Gratitude” (occurring in 57.8% of Jira data versus 2.6% in Code Reviews) demonstrate that both quantitative and qualitative dimensions are necessary to meaningfully differentiate SE datasets. Nevertheless, some features (e.g., average characters per word, number of question marks) exhibit little variance and may be omitted or combined in future iterations to streamline analysis.

Performance results reinforce that transformer-based models (SetFit, RoBERTa, ELECTRA) consistently outperform SE-specific or lexicon-based tools, in line with recent advances in trustworthy AI and requirements engineering. However, performance varies not just by platform but even within platforms, suggesting that tool selection should account for additional factors such as data source (e.g., pull requests vs. issue comments), dataset size, and labeling practices.

The prevalence of “Ambiguous” mappings in the questionnaire indicates that further refinement is needed—either through improved feature selection, normalization, or more granular platform groupings. Future work may use clustering or embedding-based similarity measures to automate dataset matching and support evolving requirements for trustworthy AI systems, as discussed in RETRAI’s scope.

Cross-platform performance gaps remain a challenge: even state-of-the-art tools perform worse on datasets that differ from their training data. A detailed correlation analysis between dataset characteristics and tool accuracy, as well as robust statistical testing of performance differences, could further substantiate these findings and provide actionable recommendations for requirements engineering in AI-based systems.

In practical terms, for direct, technical discussions (e.g., GitHub issues), lexicon-based tools may suffice due to their stability. For complex or emotionally nuanced contexts (e.g., Jira, team chats), fine-tuned machine learning models are preferable. Ultimately, selecting a tool should align with the project’s requirements for trustworthiness, explainability, and adaptability.

C. Threats to Validity

We categorize the threats to our analysis following Wohlin et al. [41].

Construct Validity. Our analysis is limited to specific projects from each platform, and alternative categorizations (e.g., by project type or granularity) could yield different results. We adopted platform definitions consistent with previous studies [13], [14], [12]. When mapping user input for statistics in the questionnaire, we used a distance measure, which does not allow for ambiguous assignment in absolute terms; setting empirical thresholds would be subjective, as no standard exists. Hyperparameters for sentiment tools were not tuned per dataset, nor were advanced analyses (e.g., k-fold cross-validation) performed, as our aim was to demonstrate the overall approach rather than optimize for marginal gains. The set of statistics chosen may not cover all possible correlations to platform characteristics, but was guided by prior work [39], and our method allows for statistics to be adapted flexibly.

The SEnti-Analyzer tool uses a majority-voting ensemble, which may behave differently from single tools; however, prior work shows only minor differences [13], [16]. A further threat is the inherent subjectivity of sentiment interpretation in SE: even gold-standard labels reflect annotator judgments that may not generalize [32], [26]. This could impact our performance comparisons and recommendations.

Internal Validity. For platform analysis, datasets were merged and sampled; differences between datasets from the same platform or varying crawling methods could introduce bias, though this risk is mitigated by source consistency. Performance was analyzed per dataset and platform. Use of third-party tools (e.g., pyspellchecker) inherits their limitations.

Conclusion Validity. Original datasets may have been filtered in ways that affect statistical analysis; we selected statistics likely unaffected by such filters. Coding of linguistic features remains partly subjective, despite calibration workshops among researchers. The identification of a “best-performing” tool is relative to each dataset, and results may not generalize beyond the contexts evaluated.

External Validity. Results are specific to the sampled data and platforms analyzed and may not generalize to all SE communication or future datasets. To reduce this threat, we included all available sentiment datasets for each platform to capture diversity within current data.

D. Future Work

Future research could extend the approach to broader social and behavioral aspects of software engineering, for example by clustering questionnaire responses to gain deeper insights into team and communication dynamics. The subjectivity and inconsistency in sentiment labeling, as highlighted in prior studies [22], [23], [42], [43], [17], [20], [26], [34], underline the value of standardized annotation processes, such as those proposed by Sun et al. [44], potentially leveraging established emotion models [45], [46]. Applying our methodology to datasets labeled under such standards could improve consistency and enable more robust cross-platform analyses.

Collaborations with industry partners could enable the inclusion of proprietary datasets, facilitating comparison between industrial and public communication channels, and covering platforms like Microsoft Teams, Slack, or Discord. The approach could also be adapted to domains beyond sentiment analysis, such as software-specific datasets.

Further enhancements may include evaluating usability and software quality aspects of sentiment analysis tools, performing correlation analyses between dataset characteristics and tool performance, or refining mapping scales (e.g., using three-point rather than four-point scales). Automating the identification of linguistic features with machine learning or clustering techniques (e.g., k-means) could make the mapping process more objective and efficient. Prioritizing features based on their predictive power for tool performance could also improve recommendation accuracy.

Importantly, the questionnaire should be validated in practice to assess its utility and accuracy in real-world scenarios. Additional work could explore a broader range of NLP statistics, focus analyses on individual datasets, and examine the influence of contextual factors such as project domain, team composition, or communication culture. These aspects may be investigated through real-world case studies or controlled user studies to further refine and validate tool recommendations.

VI. CONCLUSION

Sentiment analysis is an increasingly relevant technique in software engineering, especially for understanding collaborative communication and supporting trustworthy AI-based systems. However, our results show that sentiment analysis tool performance varies considerably depending on the linguistic and statistical characteristics of the dataset—particularly in cross-platform scenarios. Tools trained on one type of developer communication, such as GitHub, often perform suboptimally when applied to different settings like Jira due to diverse communication styles and content structures.

To address this, we presented a systematic approach that analyzes dataset characteristics and matches them with the most suitable sentiment analysis tools. By evaluating 14 tools across 10 datasets, we identified substantial differences in linguistic features and statistics between platforms. Based on these findings, we developed a questionnaire-based mapping system that allows users to describe their dataset and receive evidence-based tool recommendations—without requiring labeled data.

Our results confirm that transformer-based models such as SetFit, RoBERTa, and ELECTRA consistently achieve top performance, but also highlight that no tool is universally optimal. Tool effectiveness remains highly context-dependent, underscoring the need for careful tool selection in practice. As software engineering moves toward greater use of AI and automated analytics, aligning sentiment analysis with platform-specific characteristics is crucial for reliable and trustworthy insights.

Future research should further investigate the influence of contextual and domain-specific variables on tool performance,

and evaluate the practical impact of recommendation systems like ours in real-world software projects and requirements engineering for AI.

DATA AVAILABILITY STATEMENTS

The raw dataset containing the labeling result is available on figshare².

REFERENCES

- [1] M. Obaidi and J. Klünder, “Development and application of sentiment analysis tools in software engineering: A systematic literature review,” in *International Conference on Evaluation and Assessment in Software Engineering*, ACM, New York, NY, USA: Association for Computing Machinery, 2021.
- [2] M. Obaidi, L. Nagel, A. Specht, and J. Klünder, “Sentiment analysis tools in software engineering: A systematic mapping study,” *IST*, vol. 151, 2022.
- [3] B. Lin, N. Cassee, A. Serebrenik, G. Bavota, N. Novielli, and M. Lanza, “Opinion mining for software development: A systematic literature review,” *ACM Trans. Softw. Eng. Methodol.*, vol. 31, no. 3, mar 2022.
- [4] M. Obaidi, L. Nagel, A. Specht, and J. Klünder, “Dataset: Systematic Mapping Study on the Development and Application of Sentiment Analysis Tools in Software Engineering,” Mar. 2022.
- [5] N. Novielli, F. Calefato, and F. Lanubile, “A gold standard for emotion annotation in stack overflow,” in *MSR*, ser. MSR ’18. New York, NY, USA: Association for Computing Machinery, 2018.
- [6] F. Calefato, F. Lanubile, F. Maiorano, and N. Novielli, “Sentiment polarity detection for software development,” *Empirical Software Engineering*, vol. 23, no. 3, 2018.
- [7] M. Herrmann and J. Klünder, “From textual to verbal communication: Towards applying sentiment analysis to a software project meeting,” in *REW*. Piscataway, NJ, USA: IEEE, 2021.
- [8] M. R. Islam and M. F. Zibran, “Sentistrength-se: Exploiting domain specificity for improved sentiment analysis in software engineering text,” *JSS*, vol. 145, 2018.
- [9] T. Ahmed, A. Bosu, A. Iqbal, and S. Rahimi, “Senticr: A customized sentiment analysis tool for code review interactions,” in *ASE*. IEEE, 2017.
- [10] A. Specht, M. Obaidi, L. Nagel, M. Stess, and J. Klünder, “What is needed to apply sentiment analysis in real software projects: A feasibility study in industry,” in *Human-Centered Software Engineering*. Cham: Springer Nature Switzerland, 2024, pp. 105–129.
- [11] K. Madampe, R. Hoda, and P. Singh, “Towards understanding emotional response to requirements changes in agile teams,” in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: New Ideas and Emerging Results*, ser. ICSE-NIER ’20. New York, NY, USA: Association for Computing Machinery, 2020, p. 37–40. [Online]. Available: <https://doi.org/10.1145/3377816.3381722>
- [12] N. Novielli, F. Calefato, D. Dongiovanni, D. Girardi, and F. Lanubile, “Can we use se-specific sentiment analysis tools in a cross-platform setting?” in *Proceedings of the 17th International Conference on Mining Software Repositories*, ser. MSR ’20. New York, NY, USA: Association for Computing Machinery, 2020.
- [13] M. Obaidi, H. Holm, K. Schneider, and J. Klünder, “On the limitations of combining sentiment analysis tools in a cross-platform setting,” in *Product-Focused Software Process Improvement*, D. Taibi, M. Kuhrmann, T. Mikkonen, J. Klünder, and P. Abrahamsson, Eds. Cham: Springer International Publishing, 2022.
- [14] T. Zhang, B. Xu, F. Thung, S. A. Haryono, D. Lo, and L. Jiang, “Sentiment analysis for software engineering: How far can pre-trained transformer models go?” in *ICSME*, 2020.
- [15] B. Lin, F. Zampetti, G. Bavota, M. Di Penta, M. Lanza, and R. Oliveto, “Sentiment analysis for software engineering: How far can we go?” in *Proceedings of the 40th International Conference on Software Engineering*, ser. ICSE ’18. New York, NY, USA: Association for Computing Machinery, 2018.
- [16] G. Uddin, Y.-G. Guéhénuc, F. Khomh, and C. K. Roy, “An empirical study of the effectiveness of an ensemble of stand-alone sentiment detection tools for software engineering datasets,” *ACM Trans. Softw. Eng. Methodol.*, vol. 31, no. 3, apr 2022.

²<https://figshare.com/s/c6356cb22a874f14bd13>

- [17] L. A. Cabrera-Diego, N. Bessis, and I. Korkontzelos, "Classifying emotions in stack overflow and jira using a multi-label approach," *Knowledge-Based Systems*, vol. 195, 2020.
- [18] M. E. Whiting, I. Gao, M. Xing, N. J. Diarrassouba, T. Nguyen, and M. S. Bernstein, "Parallel worlds: Repeated initializations of the same team to improve team viability," vol. 4, no. CSCW1, May 2020.
- [19] L. Mostafa and M. Abd Elghany, *INVESTIGATING GAME DEVELOPERS' GUILT EMOTIONS USING SENTIMENT ANALYSIS*, 2018, vol. 9.
- [20] G. Uddin and F. Khomh, "Automatic mining of opinions expressed about apis in stack overflow," *IEEE Transactions on Software Engineering*, vol. 47, no. 3, 2021.
- [21] N. Novielli, F. Calefato, D. Dongiovanni, D. Girardi, and F. Lanubile, "A gold standard for polarity of emotions of software developers in github," 3 2020.
- [22] J. Ding, H. Sun, X. Wang, and X. Liu, "Entity-level sentiment analysis of issue comments," in *Proceedings of the 3rd International Workshop on Emotion Awareness in Software Engineering*, ser. SEMotion '18. New York, NY, USA: Association for Computing Machinery, 2018.
- [23] N. Intiaz, J. Middleton, P. Girouard, and E. Murphy-Hill, "Sentiment and politeness analysis tools on developer discussions are unreliable, but so are people," in *Proceedings of the 3rd International Workshop on Emotion Awareness in Software Engineering*, ser. SEMotion '18. New York, NY, USA: Association for Computing Machinery, 2018.
- [24] M. Ortu, A. Murgia, G. Destefanis, P. Tourani, R. Tonelli, M. Marchesi, and B. Adams, "The emotional side of software developers in jira," in *Proceedings of the 13th International Conference on Mining Software Repositories*, ser. MSR '16. New York, NY, USA: Association for Computing Machinery, 2016.
- [25] M. R. Islam and M. F. Zibran, "Deva: sensing emotions in the valence arousal space in software engineering text," in *Applied computing 2018*, H. M. Haddad, R. L. Wainwright, and R. Chbeir, Eds. New York, NY: Association for Computing Machinery Inc. (ACM), 2018.
- [26] M. Herrmann, M. Obaidi, L. Chazette, and J. Klünder, "On the subjectivity of emotions in software projects: How reliable are pre-labeled data sets for sentiment analysis?" *JSS*, vol. 193, 2022.
- [27] M. Thelwall, K. Buckley, and G. Paltoglou, "Sentiment strength detection for the social web," *Journal of the American Society for Information Science and Technology*, vol. 63, no. 1, 2012.
- [28] E. Biswas, M. E. Karabulut, L. Pollock, and K. Vijay-Shanker, "Achieving reliable sentiment analysis in the software engineering domain using bert," in *ICSME*, 2020.
- [29] Z. Chen, Y. Cao, X. Lu, Q. Mei, and X. Liu, "Sentimoji: An emoji-powered learning approach for sentiment analysis in software engineering," in *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2019. New York, NY, USA: Association for Computing Machinery, 2019.
- [30] M. Herrmann, M. Obaidi, and J. Klünder, "Senti-analyzer: Joint sentiment analysis for text-based and verbal communication in software projects," Leibniz University Hannover, Tech. Rep., 2022.
- [31] T. Zhang, I. C. Irsan, F. Thung, and D. Lo, "Revisiting sentiment analysis for software engineering in the era of large language models," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 3, Feb. 2025. [Online]. Available: <https://doi.org/10.1145/3697009>
- [32] M. Herrmann, M. Obaidi, and J. Klünder, "Different and similar perceptions of communication among software developers," *Information and Software Technology*, vol. 181, p. 107698, 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0950584925000370>
- [33] M. Herrmann, M. Obaidi, and J. Klünder, "Modeling communication perception in development teams using monte carlo methods," in *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering*, ser. EASE '25. Association for Computing Machinery, 2025.
- [34] M. Obaidi, M. Herrmann, L. Chazette, and J. Klünder, "Dataset: On the Subjectivity of Emotions in Software Projects: How Reliable are Pre-Labeled Data Sets for Sentiment Analysis?" Jun. 2022.
- [35] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," 2019.
- [36] L. Tunstall, N. Reimers, U. E. S. Jo, L. Bates, D. Korat, M. Wasserblat, and O. Pereg, "Efficient few-shot learning without prompts," 2022.
- [37] J. L. Fleiss, "Measuring nominal scale agreement among many raters," *Psychological Bulletin*, vol. 76, no. 5, 1971.
- [38] J. Landis and G. Koch, "The measurement of observer agreement for categorical data," *Biometrics*, vol. 33 1, 1977.
- [39] J. Klünder, J. Horstmann, and O. Karras, "Identifying the mood of a software development team by analyzing text-based communication in chats with machine learning," in *Human-Centered Software Engineering*, R. Bernhaupt, C. Ardito, and S. Sauer, Eds. Cham: Springer International Publishing, 2020.
- [40] M. Claes, M. Mäntylä, and U. Farooq, "On the use of emoticons in open source software development," in *Proceedings of the 12th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, ser. ACM Digital Library, M. Oivo, Ed. New York, NY: ACM, 2018.
- [41] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in software engineering*. Springer, 2012.
- [42] Marco Ortuand Bram Adams, Giuseppe Destefanis, Parastou Tourani, Michele Marchesi, and Roberto Tonelli, "Are bullies more productive? empirical study of affectiveness vs. issue fixing time," in *2015 IEEE/ACM 12th Working Conference on Mining Software Repositories*, 2015.
- [43] A. Murgia, P. Tourani, B. Adams, and M. Ortu, "Do developers feel emotions? an exploratory analysis of emotions in software artifacts," in *MSR'14*, ser. MSR 2014. New York, NY, USA: Association for Computing Machinery, 2014.
- [44] K. Sun, H. Gao, H. Kuang, X. Ma, G. Rong, D. Shao, and H. Zhang, "Exploiting the unique expression for improved sentiment analysis in software engineering text," in *2021 IEEE/ACM 29th International Conference on Program Comprehension (ICPC)*, 2021, pp. 149–159.
- [45] P. R. Shaver, J. C. Schwartz, D. Kirson, and C. O'Connor, "Emotion knowledge: Further exploration of a prototype approach," *Journal of Personality and Social Psychology*, vol. 52, no. 6, pp. 1061–1086, 1987.
- [46] W. G. Parrott, *Emotions in social psychology: Essential readings*. psychology press, 2001.